# Axon-X Starter Guide

operators who want to use Axon-X today without learning everything at once

---

Axon-X · http://127.0.0.1:4173 · axon-watch repo
Early bootstrap guide · 2026-07-05 · Read in layers — you do not need the whole system at once.

## Contents

# Axon-X Starter Guide

**Version:** early bootstrap (2026-07-05)
**For:** operators who want to use Axon-X today without learning everything at once
**Repo:** `axon-watch` · **URL:** `http://127.0.0.1:4173`

---

## Read this first (60 seconds)

Axon-X is the **next-generation Axon console**. It is real and usable for bounded local work, but it is **not** a full replacement for classic Axon on port **7734** yet.

At this stage, treat Axon-X as:

- a **workspace-scoped operator shell** (runs, signals, conversation, terminal, files)
- backed by three local services you start with one command
- growing by **thin verified slices**, not big-bang parity

You do **not** need to learn the whole architecture to be productive. Start with **one workspace**, **Operator mode**, and the **Command seam**.

---

## Two apps on your machine

| | Classic Axon ( `axon-local` ) | Axon-X ( `axon-watch` ) |
|---|---|---|
| URL | `http://127.0.0.1:7734` | `http://127.0.0.1:4173` |
| Start | `./start.sh` in axon-local | `./scripts/dev/up.sh` in axon-watch |
| Status | Daily driver | Early rebuild |

They can run at the same time on different ports. Planning docs still live in `axon-local/Plans/Axon-Watch/` ; implementation lives in `axon-watch` .

---

## Start and stop (the only commands you need daily)

```
cd /home/edp/axon-nvme/repos/axon-watch
npm install          # once, or after dependency changes
./scripts/dev/up.sh  # start console + control-plane + watch
```

Open: **http://127.0.0.1:4173**

Health check:

```
./scripts/dev/check-health.sh
```

Stop:

```
./scripts/dev/down.sh
```

**Ports:** 4173 (UI), 8787 (control-plane API), 8788 (watch service).

Logs if something fails: `.local/logs/console-web.log`, `control-plane.log`, `axon-watch.log`.

---

## What you see: five regions

The shell layout is **locked** — regions do not move between modes.

| Region | What it is for |
|---|---|
| **Top bar** | Identity, runtime strip, KAIRO presence chip, **Operator / IDE toggle** |
| **Left sidebar** | Workspaces (and in Operator: **Attention** toggle) |
| **Center** | Operator: **Mission Control** · IDE: Monaco editor + terminal dock |
| **Right dock** | Conversation, Command/KAIRO hero, run context |
| **Status bar** | Watch / run phase / signals glance strip |

---

## Operator vs IDE — pick one mindset

### Operator mode (default for monitoring)

Use when you want to **watch, decide, and steer** — not edit code in the center.

- Center = **Mission Control** (current run, live feed, STOP/RESUME/APPROVE/REJECT)
- Terminal dock is **collapsed by default** (reopen via chip or bottom strip)
- Left sidebar has **WORKSPACES | ATTENTION**
- Right dock is **conversation-first**

### IDE mode (default for editing)

Use when you want **files, explorer, and editor**.

- Center = Monaco editor + resizable terminal dock (visible by default)
- Left sidebar = workspaces + file explorer tree
- Right dock keeps full run / approvals / signals stack

**Toggle:** top bar **Operator / IDE** switch. Choice persists in session storage.

---

## Workspaces — what they mean today

A **workspace** is a logical context: files, terminal cwd, chat thread, and runs all scope to the selected `workspace_id`.

### Sidebar workspaces (what the UI shows)

The operator sidebar shows **seven mockup IDs**:

- `workspace_smoke` ← **start here** (acceptance / smoke workspace)
- `workspace_recsys`, `workspace_finance`, `workspace_nlp`, `workspace_cv`, `workspace_edge`, `workspace_research`

Selecting a workspace reloads: file tree, terminal session, chat thread, run context.

**Bootstrap rule:** on load, Axon-X picks active run workspace (if sidebar-visible), else **`workspace_smoke`**, else the first sidebar entry.

### Hidden / API-only workspaces (important)

The API may list **more** workspaces than the sidebar shows, including:

- `workspace_alpha`, `workspace_bootstrap` (test fixtures)
- **`workspace_axon_local`** → bound to sibling `../axon-local` repo
- **`workspace_axon_watch`** → bound to the axon-watch repo itself

These bound workspaces work for **terminal, git, files, and API** — but they are **not in the sidebar yet**. Use the API or curl until UI catalog unification lands.

Bindings file: `config/workspace-project-bindings.json`

---

## Your first 10 minutes (recommended path)

1. `./scripts/dev/up.sh`

2. Open `http://127.0.0.1:4173`
3. Confirm **`workspace_smoke`** is selected (left sidebar)
4. Stay in **Operator** mode
5. Glance at **Mission Control** (center) — idle standby is normal with no active run
6. Open **Conversation** (right dock) — empty until you send a command
7. In **Command** (bottom hero), type: `git status` → Enter
8. Read the agent reply in Conversation
9. Toggle left sidebar to **ATTENTION** — see runs / approvals / signals stack
10. Optional: click **Open terminal** in Mission Control header → run `pwd` in the PTY

You have now exercised: workspace scope, command executor, mission control, attention sidebar, and terminal attachment.

---

## Agents and runs — how work actually happens

Axon-X does not yet mirror every axon-local agent flow. Today:

### Command seam (chat → bounded executor)

Type natural commands in the **Command** hero. Supported intents:

| You type | What happens |
|---|---|
| `health` | Probes control-plane `/api/health` |
| `ls` or `list files` | Lists files in workspace root |
| `read README.md` | Reads a workspace file |
| `git status` | Runs git in workspace root (real output for bound repos) |
| `resume from review` | Resumes primary `review_ready` run for this workspace |

Unsupported text gets a helpful capability list back — not a silent failure.

### Runs (explicit execution units)

Runs have phases ( `executing` , `awaiting_approval` , `review_ready` , …) and capability flags ( `can_stop` , `can_resume` , `can_approve` , …).

**Where to act on runs:**

- **Mission Control** (Operator center): STOP, RESUME, COMPLETE, APPROVE, REJECT
- **Attention sidebar** (Operator): APPROVE / REJECT on pending items

- **Status bar / top runtime strip**: phase glance only (not primary controls)

Creating runs via API is supported ( `POST /api/runs` ); the UI surfaces active runs from canonical control-plane state.

### Conversation rehydration

After a hard reload, Conversation reloads the **latest thread for the current workspace**. Messages do not bleed across workspaces.

---

## Connecting real projects (TEST-1 slice)

Thin workspaces normally live under:

`.local/workspaces/{workspace_id}/`

**Bound workspaces** map to real directories via `config/workspace-project-bindings.json` :

```
{
  "bindings": {
    "workspace_axon_local": {
      "project_root": "../axon-local",
      "display_name": "axon-local"
    },
    "workspace_axon_watch": {
      "project_root": ".",
      "display_name": "axon-watch"
    }
  }
}
```

When bound:

- Terminal cwd = real project root (no fake isolated folder)
- `git status` in Command = real repo status
- Monaco file host reads/writes the real tree

**Safety:** paths must fall inside `AXON_WATCH_PROJECT_ROOT_ALLOWLIST` (repo root, parent directory, home — configurable).

Verify bindings:

```
curl -s http://127.0.0.1:8787/api/workspaces/workspace_axon_local | python3 -m json.tool
```

Expect `"connection_kind": "project_path"` and a resolved `project_root` .

---

## Cross-workspace handoffs (TEST-2 slice)

Record an explicit handoff when work should continue in another workspace:

```
curl -s -X POST http://127.0.0.1:8787/api/workspaces/workspace_smoke/handoffs \
  -H 'Content-Type: application/json' \
  -d '{
    "target_workspace_id": "workspace_axon_local",
    "task": "Review axon-local after my smoke test",
    "reason": "Cross-repo follow-up"
  }' | python3 -m json.tool
```

Response includes:

- persisted **`handoff`** record ( `handoff_id` , `status: recorded` )
- **`target_workspace_summary`** (connection metadata + run counts)

List handoffs:

```
curl -s http://127.0.0.1:8787/api/workspaces/workspace_smoke/handoffs | python3 -m json.too
```

**UI note:** handoffs are API-first today; switching workspace in the sidebar is still manual follow-through.

---

## Hidden features and power-user tips

### 1. Operator terminal is intentionally hidden

Operator mode stores terminal visibility separately from IDE:

- Default: **collapsed**
- Reopen: Mission Control header **Open terminal** chip, or bottom **Terminal dock · Show**
- Close: tab bar ✕ on the terminal panel

IDE mode keeps terminal visible by default and restores via status bar **TERMINAL**.

### 2. Left sidebar ATTENTION auto-focus

In Operator mode, if approvals or interruptive signals exist, the sidebar may default to **ATTENTION** instead of WORKSPACES — so you see blockers first.

### 3. Live refresh (SSE)

The shell listens to `GET /api/live/events` for refresh hints. If EventSource is unavailable, it falls back to visibility-aware polling.

### 4. Pinned workspace memory

Operator workspace selection can persist in session storage — reload may restore your last explicit pick when bootstrap rules allow.

### 5. KAIRO Briefing rhythm

`GET /api/briefing` feeds **Notice** and **Advise** strings into Mission Control (idle) and the KAIRO hero. This is presentation scaffolding — full KAIRO presence (voice, persona, watch rules) is **not** landed yet.

### 6. API catalog vs sidebar catalog

`GET /api/workspaces` is authoritative; sidebar is a trimmed operator view. Do not assume “not in sidebar” means “does not exist”.

### 7. Acceptance gates (for confidence, not daily use)

```
npm run verify:test0   # workspace_smoke acceptance
npm run verify:test1   # real project connection
npm run verify:test2   # workspace handoff slice
npm run verify         # full contract + UI build gate
```

### 8. Bound workspace git from Command

On `workspace_axon_local` (via API scope or future UI):

```
git status
```

runs against the real axon-local tree when bindings and stack are up.

---

## What to do at this early stage (honest scope)

**Do use Axon-X for:**

- exploring the new Operator / IDE shell split
- workspace-scoped terminal + file editing (IDE mode)
- bounded commands ( `git status` , file read/list, health probe)
- watching run phase + approvals + signals in Attention
- verifying slices against `workspace_smoke`
- bound-repo work via API-scoped workspaces ( `workspace_axon_local` )

**Do not expect yet:**

- full axon-local feature parity (voice, mobile cockpit, full KAIRO rules, watch connectors)

- every workspace in the sidebar (bound repos are API-visible first)
- automatic cross-workspace agent migration (handoff = record + summary, manual switch)
- production deployment / dedicated-server cutover

**When stuck, check:**

1. Is the stack up? `./scripts/dev/check-health.sh`
2. Is the right workspace selected?
3. Operator vs IDE — are you in the mode that matches your task?
4. Classic Axon still available on **7734** for daily-driver gaps

---

## Quick API reference (copy/paste)

```
 # Health
curl -s http://127.0.0.1:8787/api/health

# Workspaces
curl -s http://127.0.0.1:8787/api/workspaces | python3 -m json.tool

# Runtime truth
curl -s http://127.0.0.1:8787/api/runtime/summary | python3 -m json.tool

# Briefing (Notice / Advise)
curl -s http://127.0.0.1:8787/api/briefing | python3 -m json.tool

# Post a chat command
curl -s -X POST http://127.0.0.1:8787/api/chat/messages \
  -H 'Content-Type: application/json' \
  -d '{"workspace_id":"workspace_smoke","content":"git status"}' | python3 -m json.tool

# Workspace chat thread (rehydration)
curl -s http://127.0.0.1:8787/api/workspaces/workspace_smoke/chat/thread | python3 -m json.
```

---

## Learning in layers (don't read everything at once)

| When you want… | Read… |
|---|---|
| Daily usage | This guide |
| Deeper handbook | `docs/HOW-TO-HANDBOOK.md` |
| Mission Control UI | `docs/OPERATOR_MISSION_CONTROL_V1.md` |
| Project bindings | `docs/WORKSPACE_PROJECT_CONNECTION.md` |
| Handoffs | `docs/WORKSPACE_HANDOFF.md` |
| Cutover readiness | `docs/AXON_X_CUTOVER_TODO.md` |
| Layout rules | `docs/UI_LAYOUT_LOCK.md` |

## Troubleshooting (short)

| Problem | Fix |
|---|---|
| Port in use | `./scripts/dev/down.sh` then retry `up.sh` |
| Blank runtime / briefing | Control-plane not ready — check `.local/logs/control-plane.log` |
| Command says unsupported | See supported command table above |
| Terminal empty / wrong cwd | Confirm workspace; bound vs isolated root |
| Conversation empty after reload | Reselect workspace; check `/chat/thread` API |
| Sidebar missing axon-local | Expected — use API workspace until catalog unifies |

*Axon-X · starter guide · generated for operator onboarding · 2026-07-05*

Generated from docs/AXON-X-STARTER-GUIDE.md · Rebuild: ./scripts/docs/build-starter-guide-pdf.sh